

Contagem Paralela de Objetos em Matriz Binária

Implementação ANSI C89 (Pthreads) com Decomposição 2D e Consolidação por Union-Find

Disciplina: Sistemas Operacionais (2026/II)

Professor: Prof. Filipo Novo Mór

Instituição: PUCRS — Escola Politécnica

Autores: João Pedro Wiskow Marth, Rafael dos Reis, Guilherme Dentzien Silva & Lucas Goettert Lopes

■ Vídeo de Apresentação Gravada (YouTube): https://youtu.be/Ch_qF0VLAjM

- **Conectividade 8:**

Células de valor 1 conectadas horizontalmente, verticalmente ou pelas 4 diagonais pertencem ao mesmo objeto.

- **Matriz Binária:**

Entradas binárias de tamanho arbitrário $R \times C$ onde 0 é fundo e 1 é objeto.

- **Desafio Principal:**

Decompor o cálculo entre múltiplas unidades de execução mantendo determinismo e unificando objetos que cruzam fronteiras de partições.

- **Conformidade Estrita:**

Padrão ANSI C (C89/C90), compilação sem warnings (-Wall -Wextra -pedantic -pthread).

- **Algoritmo de 2 Passagens:**

Baseado na estrutura de dados Union-Find (Disjoint Set Union).

- **Passagem 1:**

Percorre a matriz e executa `uf_union` entre células vizinhas de valor 1 em 8 direções.

- **Passagem 2:**

Identifica raízes únicas (representantes dos componentes) associadas a células de primeiro plano.

- **Otimizações:**

Compressão de caminho (Path Compression) e união por rank garantem complexidade $O(R * C * \alpha(N)) \approx O(N)$.

- **Função de Referência:**

Serviu como oráculo de correção para validar a implementação paralela.

- **Modelo de Concorrência:**

POSIX Threads (Pthreads) e sincronização refinada por mutex (`pthread_mutex_t`).

- **Decomposição em Grade 2D:**

A matriz é particionada em sub-blocos retangulares/quadrados ($P_r \times P_c$).

- **Fase 1 (Rotulagem Local Paralela):**

Cada thread rotula exclusivamente os elementos internos da sua região sem contenção ou travas.

- **Balanceamento de Carga:**

Atribuição estática e proporcional de linhas e colunas para minimizar overhead de escalonamento.

- **Portabilidade:**

Configuração dinâmica de trabalhadores (1, 2, 4, 8 threads).

- **Problema de Fronteira:**

Um objeto pode ocupar múltiplas regiões. Somar contagens locais gera contagem duplicada incorreta.

- **Fase 2 (Stitching):**

Inspeção sistemática das bordas entre blocos adjacentes:

- **Fronteiras Horizontais:** Unificação de células 1 entre linhas `r_boundary` e `r_boundary+1`.
- **Fronteiras Verticais:** Unificação de células 1 entre colunas `c_boundary` e `c_boundary+1`.
- **Quinas / Encontro de 4 Blocos:** Tratamento de conexões diagonais que cruzam os 4 vértices adjacentes.

- **Sincronização:**

Operações de unificação entre bordas utilizam `uf_union_threadsafe` com proteção por mutex.

- **Matrizes Obrigatórias do Enunciado:**

Matriz 1 (5x5, 2x2 blocos)	3 objetos	[PASSOU]
Matriz 2 (6x8, 2x2 blocos)	4 objetos	[PASSOU]
Matriz 3 (8x8, 2x2 blocos)	5 objetos	[PASSOU]
Matriz 4 (9x12, 3x3 blocos)	6 objetos	[PASSOU]
Matriz 5 (12x12, 3x3 blocos)	7 objetos	[PASSOU]

- **Testes Adicionais:**

Validação em matrizes sintéticas gigantes (5000x5000 com 30% de densidade, 1.180.391 objetos).

- **Determinismo:**

100% de correspondência exata de resultados entre as versões sequencial e paralela em todas as repetições.

- **Ambiente Experimental:**

Apple M5 (10 núcleos lógicos, 16 GB RAM, macOS 26.6.2, Clang 21.0 C89).

- **Medições em Matriz 5000x5000 (1,18 milhão de objetos):**

Versão / Threads	Tempo (ms)	Speedup Relativo
Sequencial	265.49 ms	1.00x (Baseline)
1 Thread	295.84 ms	0.90x
2 Threads	286.06 ms	0.93x
4 Threads	338.85 ms	0.78x
8 Threads	360.30 ms	0.74x

- **Repetibilidade:**

Mediana de 5 repetições com baixo desvio-padrão.

- **Overhead de Sincronização:**

A Fase 3 realiza chamadas `uf_find_threadsafe` com trava de mutex por célula 1, gerando gargalo com muitas threads.

- **False Sharing:**

O vetor parent do Union-Find é compartilhado na mesma linha de cache (64 bytes). Writes frequentes causam cache line bouncing.

- **Lei de Amdahl:**

A Fase 2 de consolidação de fronteiras roda de forma sequencial na thread principal.

- **Otimizações Futuras:**

Union-Find lock-free baseado em rotulagem com CAS (Compare-And-Swap) ou tabelas locais de equivalência.

- **Objetivos Alcançados:**

Correção 100% comprovada com conectividade 8, tratamento rigoroso de bordas/quinas e conformidade ANSI C89.

- **Concorrência vs Paralelismo:**

Nem toda divisão paralela resulta em speedup linear sem refatoração do layout de memória para evitar contenção de travas e tráfego de cache.

- **Organização & Qualidade:**

Automação completa via Makefile, documentação em RELATORIO_TECNICO.md, gráficos de benchmark e código robusto.

Assista à Apresentação Gravada no YouTube:

https://youtu.be/Ch_qFOVLAjM

Detalhes da Gravação:

- **Plataforma:** YouTube (Vídeo Não Listado)
- **Duração:** 09:45 (dentro do limite máximo de 10 minutos)
- **Tópicos Cobertos:** Estratégia de conectividade 8, implementação sequencial vs paralela em Pthreads C89, algoritmo de stitching em 2D, demonstração prática e análise de desempenho com benchmark.